

Tensegrity Rover Simulator

Project Overview & Technical Architecture

1. What is Going On in This Repo?

This repository contains a **Browser-based 3D Physics Simulation** of a six-strut, 24-cable tensegrity rover.

Key Features:

- **Tensegrity Physics:** Accurately models a soft-robotics structure (tensegrity) navigating challenging terrain.
- **Payload Isolation:** Tests how well the outer cage protects a suspended inner core from impact shocks (e.g., keeping G-forces below 1.5 Gs).
- **Advanced Control Systems:** Integrates 8 different control algorithms (from simple LQR to complex QP-MPC and Neural Networks) ported directly from MATLAB.
- **Procedural Martian Terrain:** Generates vast, varied terrains (up to 1 km² open-world) complete with craters, ridges, and thousands of interactive boulders.

Academic Purpose:

The project demonstrates applied numerical methods, control theory, and real-time rigid-body dynamics for aerospace and robotics exploration.

2. What Technology is Currently Being Used?

The project is built to run entirely on the client-side (in the browser), ensuring high accessibility without needing a heavy backend.

Core Technologies:

- **Vanilla JavaScript (ES6+):** The entire physics engine, constraint solvers, and numerical integration (at 500 Hz) are written from scratch in pure JS. No heavy game engines (like Unity or Unreal) are used.
 - **HTML5 & CSS3:** For the user interface, real-time monitoring HUDs, charts, and configuration panels.
 - **WebGL / Three.js (Implicit):** For rendering the 3D procedural albedo maps, zenith shadows, and instanced meshes of the thousands of Martian rocks.
 - **Linear Algebra / Math Modules:** Custom implementations of Riccati equations, Finite-difference Jacobians, and Matrix solvers to support the advanced control algorithms (e.g., LQR, MPC).
 - **Node.js:** Used strictly as a local development server and for running the automated test runner framework (`node --test`).
-

3. What Technology Will Be Used in the Future?

As the complexity of the simulation grows, the architecture will evolve to adopt modern high-performance computing standards.

Future Roadmap & Tech Stack:

- **WebAssembly (Wasm):** Porting the core 500 Hz physics engine and matrix solvers from JavaScript to Rust or C++ compiled to WebAssembly. This will drastically reduce computational overhead and prevent thermal throttling.
 - **WebGPU (Compute Shaders):** Moving collision detection (checking 40,000+ objects) and procedural terrain generation from the CPU to the GPU for massive parallel processing.
 - **TensorFlow.js / PyTorch Integration:** Transitioning from hard-coded neural net geometries to live reinforcement learning (RL). Agents will train inside the browser or connect via WebSockets to a Python backend to learn optimal locomotion dynamically.
 - **ROS (Robot Operating System) Bridge:** Connecting the browser simulation to physical hardware. The web app will act as a digital twin, sending validated control commands directly to a real tensegrity rover via ROS WebSockets.
-

Summary

- **Current State:** A highly optimized, dependency-free JavaScript physics and control theory simulator.
- **Immediate Goal:** Validate advanced MATLAB control logic (MPC, iLQR) in real-time environments.
- **Future Vision:** A WebAssembly/WebGPU powered digital twin capable of training AI and controlling physical space exploration robots.